

Comparison in C++: Basic Facilities

Committee: ISO/IEC JTC1 SC22 WG21 EWG Evolution

Document Number: P0474r0

Title: Comparison in C++ Basic Facilities

Date: 2016-10-15

Authors: Lawrence Crowl

Reply To: Lawrence Crowl, Lawrence@Crowl.org

Audience: EWG Evolution

Abstract

We propose functions that explicitly implement partial, weak and total orders on basic types. This proposal is the first step towards full implementation of explicit order requirements in C++.

Contents

[Introduction](#)

[Wording](#)

[?? Order \[order\]](#)

[??1 General \[order.general\]](#)

[??2 Enumerations \[order.enum\]](#)

[??3 Order Functions \[order.function\]](#)

[??4 Boolean Functions \[order.boolean\]](#)

[??5 Specializations \[order.special\]](#)

Introduction

In [P0100r1 Comparison in C++](#), we outlined an approach to ensuring that standard algorithm ordering requirements are routinely met. This paper proposes the first step in that approach, the definition of order functions for basic types.

Wording

The wording is as follows.

?? Order [order]

Add a new section.

??1 General [order.general]

Add a new section.

Order is defined by arithmetic relations of equivalence and precedence.

All equivalence and precedence relations are transitive ($a ? b$ and $b ? c$ implies $a ? c$)

All equivalence relations are reflexive ($a \sim a$) and symmetric ($a \sim b$ implies $b \sim a$). An *equality* (or congruence) relation is an equivalence relation that is also substitutable ($a = b$ implies $f(a) = f(b)$).

Precedence is irreflexive (not $a < a$), i.e. an element is not less than itself. Precedence is also asymmetric ($a < b$ implies not $b < a$). These axioms together constitute a *partial precedence*. A *weak precedence* also partitions the elements into *ordered equivalence classes*, that is being *unordered* with respect to an element is a transitive relation ($a < b$ implies $a < c$ or $c < b$). A *total precedence* is trichotomous (exactly one of $a < b$, $a = b$, $a > b$) where equality is defined above.

The precedence and equivalence relations defined above provide for three classes of *order*, partial, weak and total. A object is *less than* another when it *precedes* the other. A object is *greater* than another when the other *precedes* it. Otherwise, the objects are *unordered*, *equivalent*, or *equal* according to whether the precedence relation is partial, weak, or total.

A boolean function g refines a boolean function f when for every a and b , $f(a,b)$ implies $g(a,b)$. That is, if a call to f is true, so is the corresponding call to g .

An order is *consistent* with another order when the its precedence relation refines the other's precedence relation.

2 Enumerations [order.enum]

Add a new section.

```
enum class partial_ordering { less, unordered, greater };
enum class weak_ordering { less, equivalent, greater };
enum class total_ordering { less, equal, greater };
```

The enumerations `partial_ordering`, `weak_ordering`, and `total_ordering` describe the order between two objects as defined in [order.general].

3 Order Functions [order.function]

Add a new section.

```
template <typename T>
partial_ordering partial_order(const T& a, const T& b);
```

Returns:

The partial order between the two objects.

Constraints:

The order should be consistent with that implied by the comparison operators, or vice-versa.

```
template <typename T>
weak_ordering weak_order(const T& a, const T& b);
```

Returns:

The weak order between the two objects.

Constraints:

The order shall be consistent with that provided by `partial_order`. The order should be consistent with that implied by the comparison operators, or vice-versa.

```
template <typename T>
total_ordering total_order(const T& a, const T& b);
```

Returns:

The total order between the two objects.

Constraints:

The order shall be consistent with that provided by `weak_order`. The order should be consistent with that implied by the comparison operators.

4 Boolean Functions [order.boolean]

Add a new section.

Boolean functions provide potentially more efficient conditional execution.

```
template <typename T>
```

```
bool partial_less(const T& a, const T& b);
```

Returns:

```
as if partial_ordering::less == partial_order(a, b)
```

```
template <typename T>
```

```
bool weak_less(const T& a, const T& b);
```

Returns:

```
as if weak_ordering::less == weak_order(a, b)
```

```
template <typename T>
```

```
bool total_less(const T& a, const T& b);
```

Returns:

```
as if total_ordering::less == total_order(a, b)
```

```
template <typename T>
```

```
bool partial_unordered(const T& a, const T& b);
```

Returns:

```
as if partial_ordering::unordered == partial_order(a, b)
```

```
template <typename T>
```

```
bool weak_equivalent(const T& a, const T& b);
```

Returns:

```
as if weak_ordering::equivalent == weak_order(a, b)
```

```
template <typename T>
```

```
bool total_equal(const T& a, const T& b);
```

Returns:

```
as if total_ordering::equal == total_order(a, b)
```

?.?.5 Specializations [order.special]

Add a new section.

The implementation shall provide specializations for all integral, floating-point and pointer types. A negative zero shall be total_ordering::less than a positive zero. If for a type T, std::numeric_limits<T>::is_iec559 is true, then the corresponding total_order function shall match that of the totalOrder function in IEC 60559.